

PROGETTAZIONE DI SISTEMI DIGITALI

Michele De Fusco, Luigi Lo Bianco, Stefano Scarcelli, Costanza Ferrari

Relazione del progetto

Indice

1	Introduzione	2
2	Il moltiplicatore di Booth	3
2.1	Architettura del componente	3
2.2	Implementazione	5

Capitolo 1

Introduzione

La traccia richiedeva l'implementazione di un circuito per il filtraggio di un'immagine greyscale di almeno 32x32 pixel rappresentati su 8 bit unsigned.

Il filtro deve essere composto da 3x3 coefficienti (scelti in modo arbitrario dal gruppo di lavoro) rappresentati su 4 bit in complemento a 2.

La finestra di convoluzione ha la stessa dimensione del filtro, quindi 3x3 e scorre lungo tutta l'immagine per filtrare un pixel alla volta.

L'operazione di filtraggio deve essere condotta utilizzando un anchor point centrale quindi il pixel che subisce l'operazione di filtraggio è quello che si trova al centro della finestra.

L'immagine viene gestita attraverso il Buffer Line che restituisce i pixel di 3 righe allineati, pronti per essere inviati al moltiplicatore.

I buffer e i vettori vengono inizializzati a 0 per garantire lo zero padding laddove ce n'è bisogno, quindi lungo i bordi dell'immagine.

Il filtraggio vero e proprio avviene attraverso l'operazione di convoluzione: i pixel dell'immagine vengono moltiplicati con il coefficiente corrispondente del filtro e infine tutti i prodotti parziali vengono sommati insieme. Il risultato sarà il pixel centrale filtrato con il Filtro Gaussiano e Gaussian Blur che ha come coefficienti:

2	1	2
2	4	2
1	2	1

Per il testing abbiamo optato per l'immagine "Lena" che è stata processata con un semplice script matlab per estrarre le componen

Capitolo 2

Il moltiplicatore di Booth

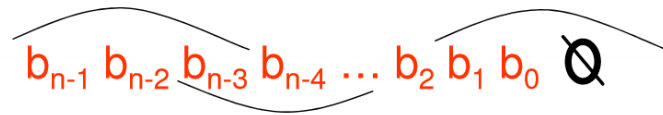
2.1 Architettura del componente

Il moltiplicatore di Booth funziona in modo diverso rispetto al moltiplicatore "carta e penna".

Se si fa distinzione fra il moltiplicando (il primo termine della moltiplicazione) e il moltiplicatore (il secondo termine della moltiplicazione) il funzionamento del moltiplicatore è incentrato sulla suddivisione in gruppi dei bit che compongono il moltiplicatore.

La suddivisione in gruppi parte dal bit più significativo e ogni gruppo deve avere un bit in comune con il gruppo successivo. Tutti i gruppi devono essere completi, quindi se restano posti vuoti nel gruppo dei bit meno significativi, questi dovrebbero essere colmati dagli zeri.

Viene definita un modo di codifica che "traduce" i gruppi dei bit in numeri signed che verranno moltiplicati (al posto di tutto il moltiplicatore) al moltiplicando.



Questa codifica sarà ciò che ci permetterà di svolgere il prodotto: quando i gruppi sono più grandi oppure non si usano gruppi ma si codifica l'intero

moltiplicando il trucco è provare a sfruttare i prodotti "immediati" con i bit, che sono quindi quelli per 0, +1, -1, +2, -2, +4, -4 e così via.

Questi prodotti sono semplici da ottenere:

- Prodotto per 0 => Il prodotto parziale avrà tutti i bit a 0;
 - Prodotto per 1 => Il prodotto parziale sarà uguale al moltiplicando;
 - Prodotto per -1 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit e a cui viene sommato 1;
 - Prodotto per 2 => Il prodotto parziale sarà uguale al moltiplicando che subisce uno shift sinistro;
 - Prodotto per -2 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e uno shift sinistro;
 - Prodotto per 4 => Il prodotto parziale sarà uguale al moltiplicando che subisce due shift sinistri;
 - Prodotto per -4 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e due shift sinistri;
- e così via.

La codifica di ogni gruppo di bit del moltiplicatore verrà poi moltiplicata con tutti i bit del moltiplicando determinando dei prodotti parziali.

Questi termini dovranno essere prima moltiplicati per il relativo peso del gruppo di cui fanno parte e poi sommati fra loro.

Il peso di ogni gruppo è determinato da quello del bit centrale, in base al peso di ogni gruppo si fanno tanti shift sinistri quanti ne richiede la potenza del 2 relativa al bit e poi si può passare ai prodotti veri e propri fra il moltiplicando e i termini codificati che corrispondono al moltiplicatore.

Se il moltiplicando e il moltiplicatore sono composti rispettivamente da m ed n bit il risultato del prodotto andrà espresso in $m + n$ bit.

E' importante ricordare che i prodotti parziali devono essere estesi con segno in modo tale che $\# \text{ shift sinistri} + \# \text{ bit del prodotto parziale} + \# \text{ bit dell'estensione con segno} = m + n$.

Dopo aver raggiunto il numero di bit corretti si può passare a sommare tutti i prodotti parziali e si avrà ottenuto il risultato del prodotto iniziale.

2.2 Implementazione

Il codice del componente booth_multiplier da noi implementato funziona nel seguente modo:

1. Prende in ingresso (oltre ai segnali clock, reset e valid utili per la sincronizzazione del circuito) i 9 segnali $P_{x,y}$ con dimensione 8 bit che rappresentano le componenti in greyscale dell'immagine e i 9 segnali $F_{x,y}$ con dimensione 4 bit che rappresentano i coefficienti del filtro. Restituisce 9 segnali $M_{x,y}$ che rappresentano le componenti $P_{x,y} * F_{x,y}$ con stesse x e y rappresentati su $8+4=12$ bit.
2. Si va a codificare il filtro secondo tutte le combinazioni entranti possibili:

Coefficienti del fitro	Numero da codificare	A	B
0000	0	0	0
0001	1	1	0
0010	2	-2	4
0011	3	-1	4
0100	4	0	4
0101	5	1	4
0110	6	-2	8
0111	7	-1	8
1000	-8	0	-8
1001	-7	1	-8
1010	-6	-2	-4
1011	-5	-1	-4
1100	-4	0	-4
1101	-3	1	-4
1110	-2	-2	0
1111	-1	-1	0

3. Dalla tabella si notano due coefficienti: A e B. Dato l'elevato numero dei casi possibili diventa complesso offrire una codifica diretta quindi si utilizzano A e B che vengono ciascuno moltiplicati per la componente dell'immagine corrispondente (che quindi subisce shift sinistri e/o negazione) e vengono salvati in $P_P_A_{x,y}$ e $P_P_B_{x,y}$.

4. Dato il numero di bit che deve essere raggiunto alla fine del moltiplicatore (12), dopo aver effettuato le operazioni necessarie, se restano posti non occupati deve essere fatto uno zero padding.

Questa operazione all'interno del codice risulta in un case statement in cui si codificano le componenti $P_P_A_{x,y}$ e $P_P_B_{x,y}$ in tutti i 16 casi possibili e si effettua lo zero padding laddove è necessario.

5. L'ultimo passaggio è la somma fra le componenti $P_P_A_{x,y}$ e $P_P_B_{x,y}$ calcolata per tutti gli ingressi attraverso dei semplici circuiti sommatori Ripple Carry.